

Seaborn

for Data Science

Statistical Data Visualization in Python – Concepts, Plots & Practicals

Prepared by

Kabir Roy

Registration No.: **23BCE10815**

B.Tech CSE, VIT Bhopal University

Course: Foundations of Data Science

Contents

1. Introduction to Seaborn	2
1.1 Why Seaborn?	2
1.2 Setting Up	2
1.2.1 Themes	2
1.3 First Plot: A Line Plot	2
2. Working with Built-in Datasets	4
2.1 Listing Available Datasets	4
2.2 Loading the <code>tips</code> Dataset	4
3. Core Seaborn Plots	5
3.1 1. Line Plot	5
3.2 2. Scatter Plot	5
3.3 3. Bar Plot	6
3.4 4. Box Plot	7
3.5 5. Heatmap (Pivoted Data)	7
3.6 Using Your Own DataFrame	8
4. Seaborn vs Matplotlib: Summary	10
5. Extra: Seaborn Essentials for Data Science	11
5.1 Categories of Seaborn Plots	11
5.2 Distribution Plots	11
5.3 Violin and Swarm Plots	11
5.4 Pair Plots – Quick Exploratory Data Analysis	11
5.5 Joint Plots	12
5.6 Regression Plots	12
5.7 Color Palettes	12
5.8 FacetGrid – Multi-panel Plots	13
5.9 Correlation Heatmaps – A Very Common DS Use Case	13
5.10 Saving Figures	13
5.11 Quick Reference: Most-Used Parameters Across Plots	13

1. Introduction to Seaborn

Seaborn is a Python data visualization library built on top of Matplotlib. It is designed to make statistical graphics easier to create and far more attractive by default.

Why Seaborn?

- Matplotlib is powerful but very **low-level** – every detail must be configured manually.
- Seaborn adds **high-level features**: automatic styling, built-in themes, smart color palettes, and tight integration with pandas DataFrames.
- Seaborn ships with several **built-in datasets** (`tips`, `flights`, `iris`, etc.) useful for practice and demonstration.
- It makes otherwise complex plots – boxplots, violin plots, heatmaps, pairplots – achievable in a single line of code.

• Tip

In short, Seaborn gives you:

- Less code to write
- Better-looking graphs out of the box
- Easy, native handling of pandas DataFrames

Setting Up

The standard import convention pairs Seaborn with Matplotlib and NumPy:

In []:

```
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
```

> Themes

Seaborn lets you switch the overall look of every plot with one function call:

In []:

```
sns.set_theme(style="darkgrid")
```

Common style options for `set_theme()`:

Style	Description
darkgrid	Gray background with white grid lines (default)
whitegrid	White background with gray grid lines
dark	Gray background, no grid
white	Plain white background, no grid
ticks	White background with axis tick marks

First Plot: A Line Plot

A simple line plot demonstrates how little code Seaborn needs to produce a styled chart:

In []:

```
x = np.array([1,2,3,4,554,332,53,234,90,32,123,3,222,34,234,34])
y = np.sin(x)
sns.lineplot(x=x, y=y)
plt.title("Beautiful Line Plot")
plt.show()
```

Out[]:

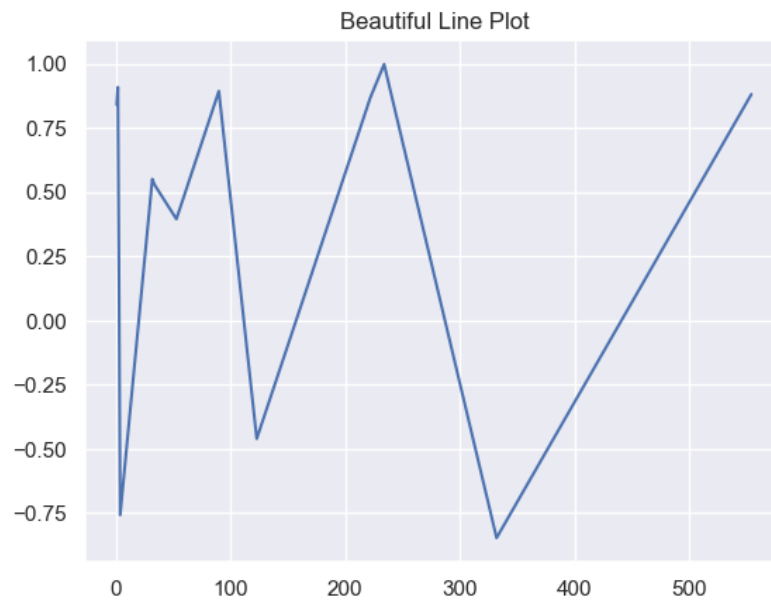


Figure 1: Line plot of $\sin(x)$ over an unsorted array, generated with the `darkgrid` theme.

*** Insight**

Because `x` is not sorted, the line plot connects points in array order rather than numeric order – this is why the line appears to zig-zag. `lineplot()` does not sort data automatically.

2. Working with Built-in Datasets

Seaborn bundles a small collection of clean, ready-to-use datasets, which is extremely convenient for learning and quick demonstrations.

Listing Available Datasets

In []:

```
print(sns.get_dataset_names())
```

Out[]:

```
['anagrams', 'anscombe', 'attention', 'brain_networks', 'car_crashes',  
'diamonds', 'dots', 'dowjones', 'exercise', 'flights', 'fmri', 'geyser',  
'glue', 'healthexp', 'iris', 'mpg', 'penguins', 'planets', 'seaice',  
'taxis', 'tips', 'titanic']
```

Loading the tips Dataset

In []:

```
tips = sns.load_dataset("tips")  
tips.head()
```

Columns in the tips dataset:

Column	Description
total_bill	Total bill amount (float)
tip	Tip amount (float)
sex	Gender of the customer (category)
smoker	Whether the customer smokes (category)
day	Day of the week (category)
time	Lunch or Dinner (category)
size	Number of people at the table (int)

244 rows, 7 columns, no missing values – confirmed via `tips.info()`.

3. Core Seaborn Plots

This section walks through the five core plot types practiced on the `tips` and `flights` datasets, followed by a custom DataFrame example.

1. Line Plot

In []:

```
tips = sns.load_dataset('tips')
sns.lineplot(x="total_bill", y="tip", data=tips)
plt.title('Line Plot Example')
plt.show()
```

Out []:

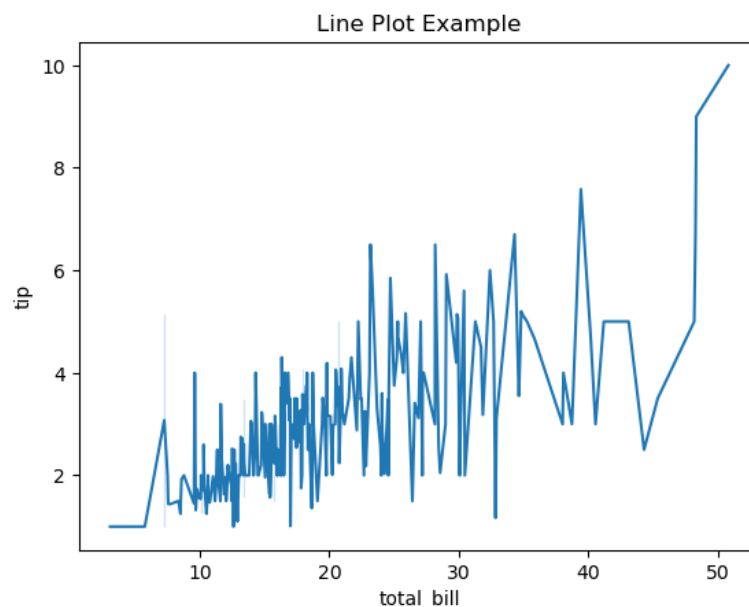


Figure 2: Line plot of tip vs. total bill, with the shaded band showing the confidence interval.

• Tip

The `data=` parameter tells Seaborn which DataFrame to pull the `x` and `y` column names from – you pass column names as strings instead of raw arrays.

2. Scatter Plot

In []:

```
sns.scatterplot(x="total_bill", y="tip", data=tips, hue="time")
plt.title('Scatter Plot with Color by Time')
plt.show()
```

Out []:

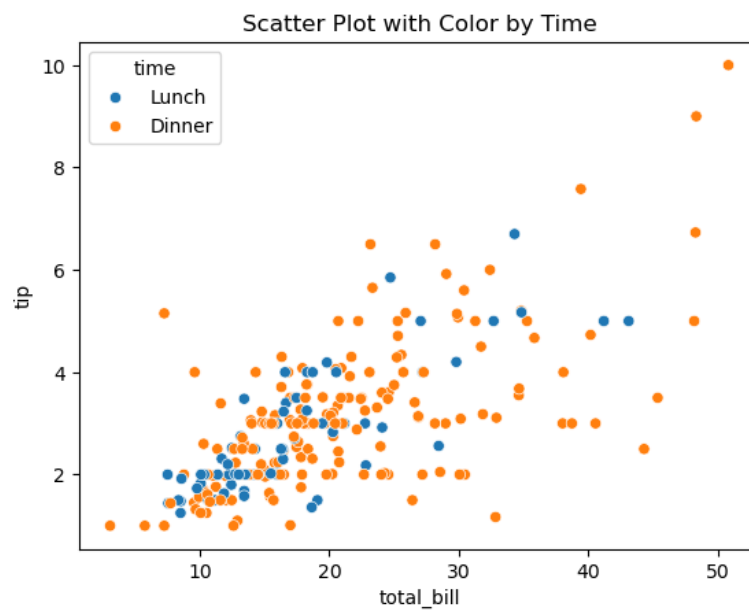


Figure 3: Scatter plot of tip vs. total bill, colored by meal time.

*** Insight**

The `hue` parameter colors data points based on the values of another column. Here, `hue="time"` automatically assigns one color to *Lunch* and another to *Dinner*, and adds a legend – no manual color mapping required.

3. Bar Plot

In []:

```
sns.barplot(x="day", y="total_bill", data=tips)
plt.title('Average Bill per Day')
plt.show()
```

Out[]:

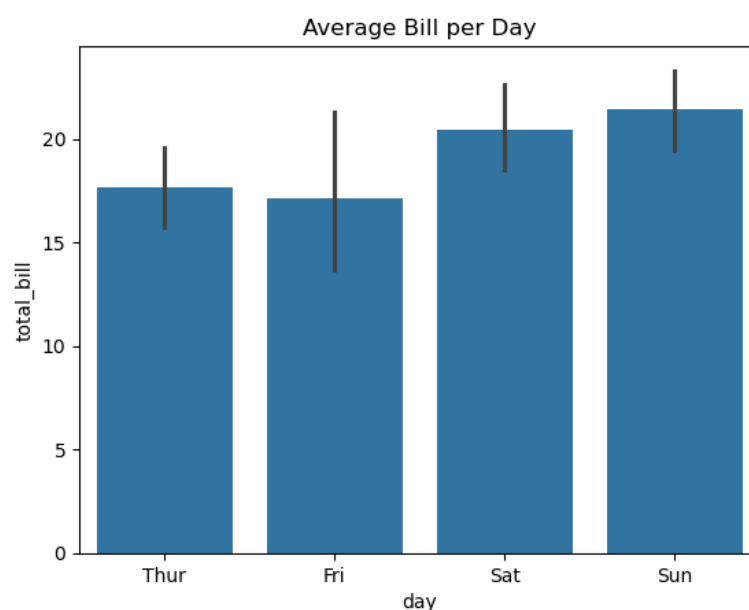


Figure 4: Bar plot showing average total bill per day, with error bars representing the confidence interval.

4. Box Plot

In []:

```
sns.boxplot(x="day", y="total_bill", data=tips)
plt.title('Boxplot of Total Bill per Day')
plt.show()
```

Out[]:

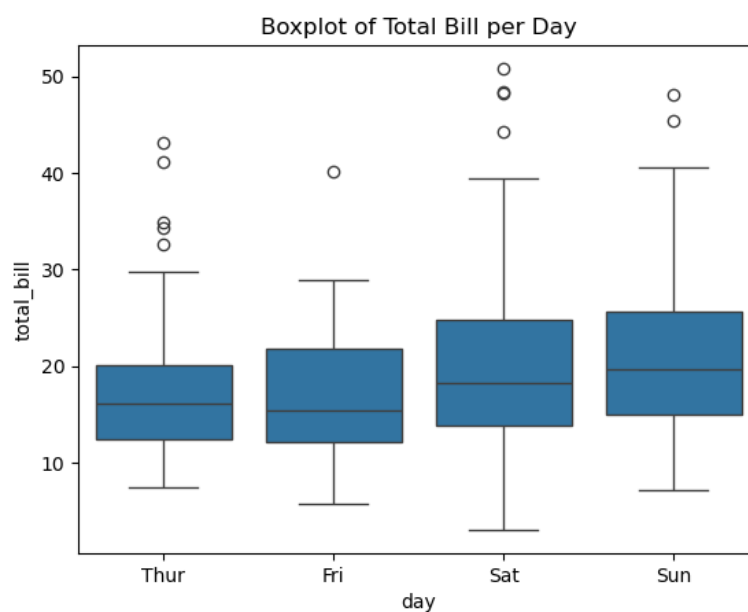


Figure 5: Box plot of total bill distribution per day, showing median, quartiles, and outliers.

• Tip

Box plots are excellent for spotting **outliers** and comparing the **spread** of a numeric variable across categories at a glance.

5. Heatmap (Pivoted Data)

In []:

```
flights = sns.load_dataset('flights')

pivot_table = flights.pivot(index="month", columns="year", values="
    passengers")

sns.heatmap(pivot_table, annot=True, fmt="d", cmap="YlGnBu")
plt.title('Heatmap of Passengers')
plt.show()
```

Out[]:

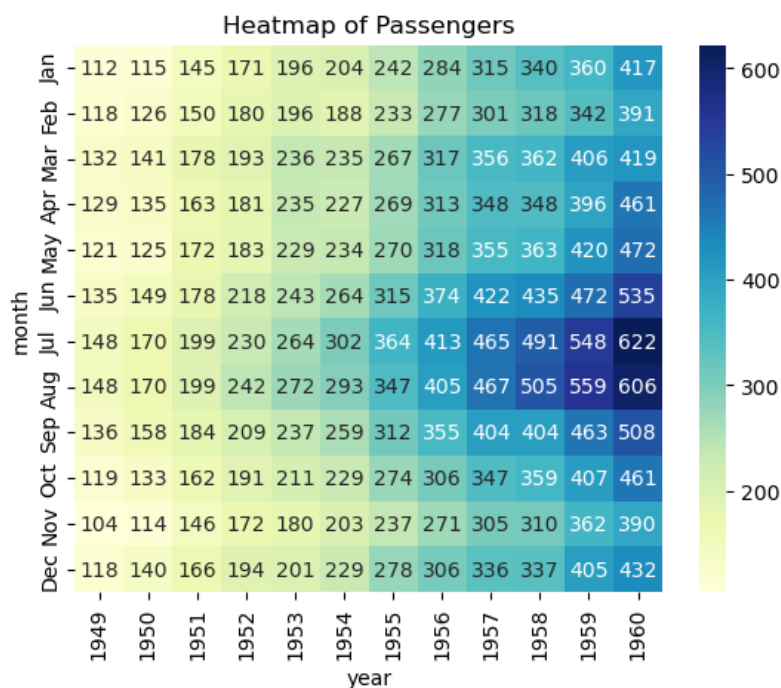


Figure 6: Heatmap of monthly flight passenger counts by year, annotated with exact values.

Key `heatmap()` parameters used:

Parameter	Description
<code>annot=True</code>	Writes the actual data value inside each cell
<code>fmt="d"</code>	Formats annotations as integers
<code>cmap="YlGnBu"</code>	Color map: Yellow → Green → Blue gradient

* Insight

Heatmaps require a **2D matrix**, not a flat DataFrame. This is why `flights.pivot()` is used first – it reshapes the long-format data so that `month` becomes rows, `year` becomes columns, and `passengers` fills the grid.

Using Your Own DataFrame

Seaborn works just as well with manually constructed DataFrames, not only its built-in datasets:
In []:

```
import pandas as pd

df = pd.DataFrame({
    "age": [22, 25, 47, 52, 46, 56, 55, 60, 34, 43],
    "salary": [25000, 27000, 52000, 60000, 58000, 62000, 61000, 65000,
               38000, 45000],
    "gender": ["M", "F", "M", "F", "F", "M", "M", "F", "F", "M"]
})

sns.scatterplot(x="age", y="salary", hue="gender", data=df)
plt.title('Salary vs Age Scatter Plot')
plt.show()
```

Out[]:

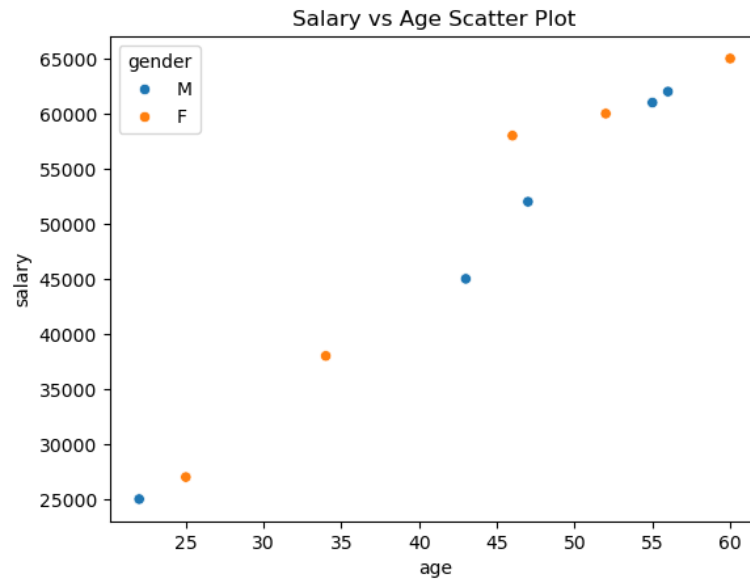


Figure 7: Scatter plot of salary vs. age on a custom DataFrame, colored by gender.

4. Seaborn vs Matplotlib: Summary

Feature	Matplotlib	Seaborn
Default Styles	Basic	Beautiful
Syntax Level	Low	High
Works with DataFrames	Manual	Easy
Plotting Complex Graphs	Tedious	Very Easy

- **Tip**

Final words: Seaborn makes data visualization faster, prettier, and smarter. You should still know Matplotlib basics for fine-tuning plots – in real-world data science work, it's common to start with Seaborn for speed, then customize with Matplotlib for final polish.

5. Extra: Seaborn Essentials for Data Science

This section covers additional Seaborn concepts and functions that are commonly required in data science work but weren't covered in the practical notebooks – useful for interviews, assignments, and EDA workflows.

Categories of Seaborn Plots

Seaborn organizes its plotting functions into a few conceptual families:

Category	Functions
Relational	scatterplot(), lineplot(), relplot()
Distribution	histplot(), kdeplot(), displot(), rugplot()
Categorical	barplot(), boxplot(), violinplot(), stripplot(), swarmplot(), countplot()
Matrix	heatmap(), clustermap()
Multi-plot grids	pairplot(), jointplot(), FacetGrid
Regression	regplot(), lmplot()

Distribution Plots

In []:

```
sns.histplot(data=tips, x="total_bill", kde=True)
sns.kdeplot(data=tips, x="total_bill", hue="time", fill=True)
```

• Tip

`histplot(kde=True)` overlays a smooth density curve on top of the histogram bars – very useful for quickly checking whether a numeric column looks normally distributed, skewed, or multimodal.

Violin and Swarm Plots

In []:

```
sns.violinplot(x="day", y="total_bill", data=tips)
sns.swarmplot(x="day", y="total_bill", data=tips, color="black", size=3)
```

* Insight

A **violin plot** combines a box plot with a kernel density estimate, showing the full shape of the distribution – not just summary statistics. **Swarm plots** place every individual data point without overlap, which is great for smaller datasets where showing each observation matters.

Pair Plots – Quick Exploratory Data Analysis

In []:

```
sns.pairplot(tips, hue="time")
```

• Tip

`pairplot()` is one of the most powerful EDA shortcuts in Seaborn: it plots every numeric column against every other numeric column in a grid, with histograms on the diagonal. One line gives you a full overview of relationships in a dataset.

Joint Plots

In []:

```
sns.jointplot(x="total_bill", y="tip", data=tips, kind="hex")
```

Common kind values for `jointplot()` / `relplot()` / `displot()`:

kind	Effect
scatter	Default; plain scatter with marginal histograms
kde	Smooth 2D density contours
hex	Hexagonal binning, useful for large datasets
reg	Adds a regression line with confidence band

Regression Plots

In []:

```
sns.regplot(x="total_bill", y="tip", data=tips)
sns.lmplot(x="total_bill", y="tip", data=tips, hue="smoker")
```

* Insight

`regplot()` fits and draws a single regression line. `lmplot()` is a figure-level version that can split the regression across multiple facets (rows/columns/hue) – e.g. separate regression lines for smokers vs. non-smokers.

Color Palettes

In []:

```
sns.set_palette("pastel")
sns.color_palette("viridis", as_cmap=True)
```

Useful built-in palettes:

Palette	Best for
deep, muted, pastel	General categorical data (Seaborn defaults)
viridis, magma, YlGnBu	Sequential numeric data, heatmaps
coolwarm, RdBu	Diverging data (e.g. correlations from -1 to 1)
Set2, tab10	Distinct categories with many levels

FacetGrid – Multi-panel Plots

In []:

```
g = sns.FacetGrid(tips, col="time", row="smoker")
g.map(sns.scatterplot, "total_bill", "tip")
```

• Tip

FacetGrid automatically creates a grid of subplots split by one or two categorical variables, then applies the same plotting function to each subset – ideal for comparing patterns across groups (e.g. smoker vs non-smoker, lunch vs dinner) without manual subplot loops.

Correlation Heatmaps – A Very Common DS Use Case

In []:

```
corr = tips.corr(numeric_only=True)
sns.heatmap(corr, annot=True, cmap="coolwarm", vmin=-1, vmax=1)
```

* Insight

This is one of the most frequently used Seaborn patterns in real data science projects: visualizing a correlation matrix to quickly spot strongly related (or unrelated) numeric features before modeling.

Saving Figures

In []:

```
fig = sns.boxplot(x="day", y="total_bill", data=tips)
fig.figure.savefig("boxplot.png", dpi=300, bbox_inches="tight")
```

• Tip

Always set `dpi=300` and `bbox_inches="tight"` when saving figures for reports or papers – this avoids blurry images and cropped axis labels.

Quick Reference: Most-Used Parameters Across Plots

Parameter	Meaning
data	DataFrame to plot from
x, y	Column names for the axes
hue	Column used to color-split the data
style	Column used to vary marker/line style
size	Column used to vary point/line size
col, row	Column(s) used to split into facets (figure-level plots)
palette	Color scheme to use
ci / errorbar	Confidence interval / error bar method

End of Notes – Seaborn for Data Science